

NVIDIA NCCL与分布式训练



限制并行应用程序的可扩展性的因素

1. 并行计算任务的效率
 - a. 并发度
 - b. 分配给每个处理器的工作量
2. 任务之间的通信成本
 - a. 通讯的数量
 - b. 通信与计算的重叠度

1.Nvidia NCCL介绍

NCCL是Nvidia Collective multi-GPU Communication Library的简称，发音为“Nickel”，是一个多GPU集群通信原语库。Nvidia做了很多优化，以在PCIe、Nvlink、InfiniBand上实现较高的通信速度。NCCL的特点为轻量级，仅依赖于通常的C++ 和 CUDA库，NCCL可以部署在单进程或多进程应用程序中，透明地处理进程间通信。

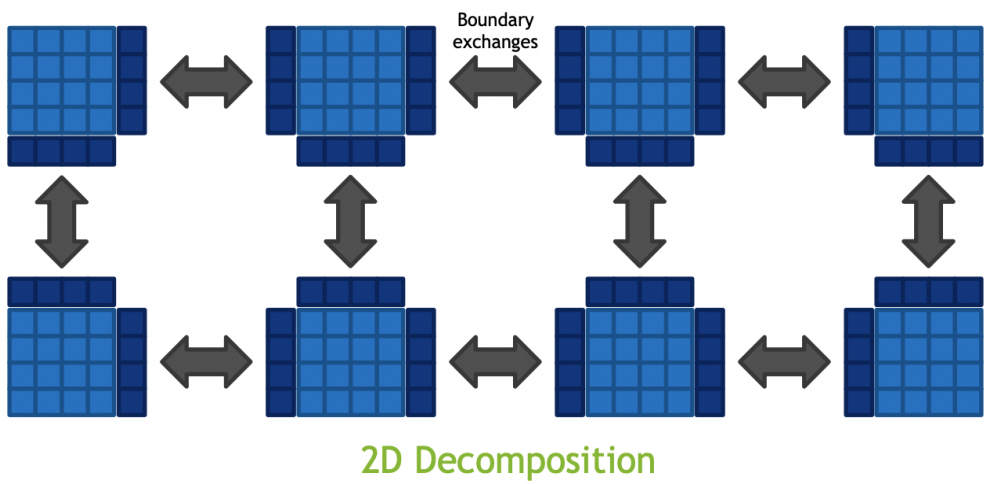
1.1 通讯方式

并行任务的通信一般可以分为P2P（Point-to-point communication）和集群通讯（Collective communication）模式。

- P2P通信这种模式只有一个sender和一个receiver，实现起来比较简单
- Collective communication包含**多个sender多个receiver**，一般的通信原语包括
 - broadcast：一个sender的数据，给**所有**节点发送**相同**数据【1对N通讯】
 - scatter：一个sender的数据，给**所有**节点发送**部分**数据【1对N通讯】
 - gather：一个receiver的数据，接收**所有**节点发送来的**部分**数据，组合成完整数据【N对1通讯】
 - all-gather：会收集**所有数据**到**所有进程**上，相当于scatter + broadcast【N对N通讯】
 - reduce：从多个sender那里接收数据，最终combine到**一个**节点上【N对1通讯】
 - all-reduce：从多个sender那里接收数据，最终combine到**每一个**节点上N对N通讯】
 - all-to-all：all-to-all是多对多通讯，相当于gather + broadcast组合。【N对N通讯】
 - scan：所有节点围成一个单向圈，每个节点依次接收上一个节点的数据并增加自己的数据。【N-1次通讯】
 - ex-scan:所有节点围成一个单向圈，每个节点依次接收上一个节点的数据并排除自己的数据。【N-1次通讯】

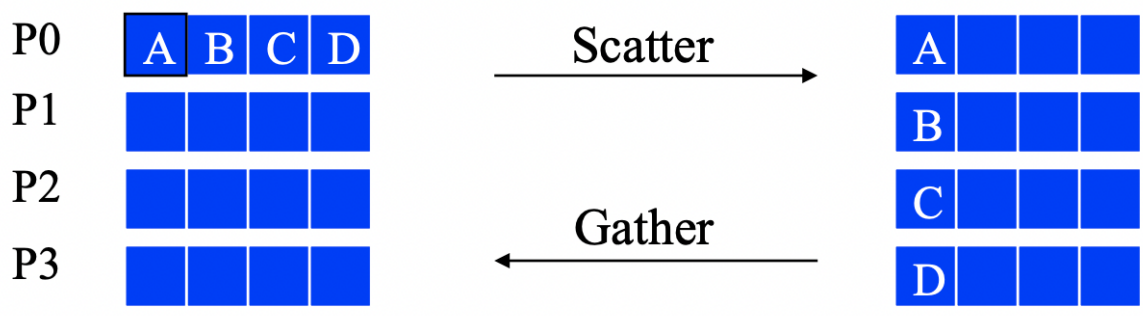
- reduce-scatter：所有节点从多个sender那里接收部分数据，combine到各自的节点上【N对N通讯】

P2P通讯模式， HPC 中最常见的模式， 通常与最近的邻居进行通信， 如下图所示：

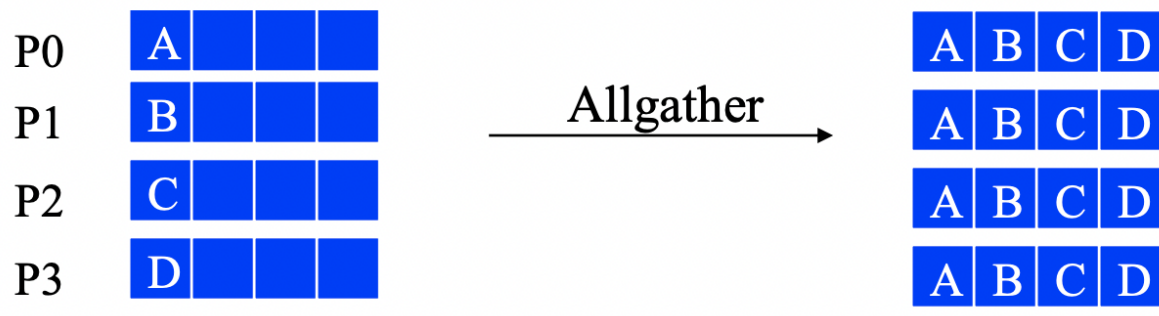


在集群通讯（Collective communication）模式下， 包含多个sender和多个receiver， 每种通讯原语在单机4卡GPU下表现如下：

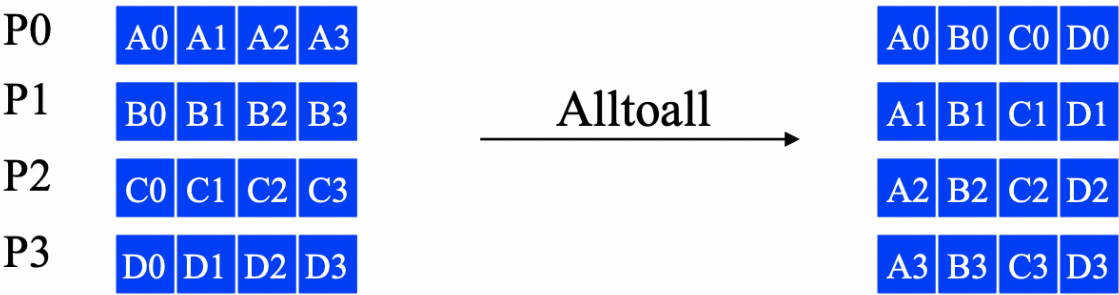
scatter与gather关系：



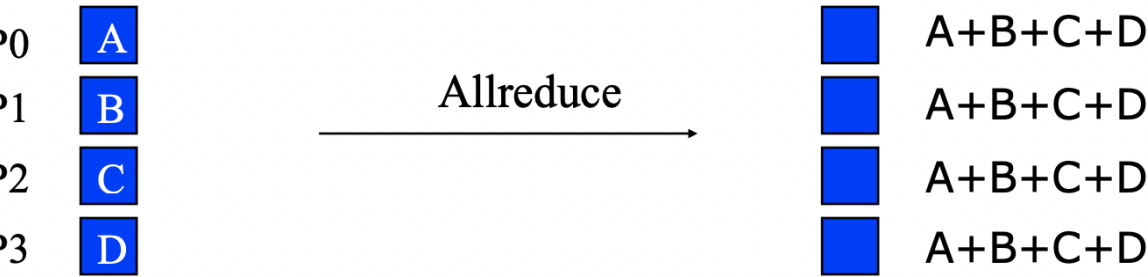
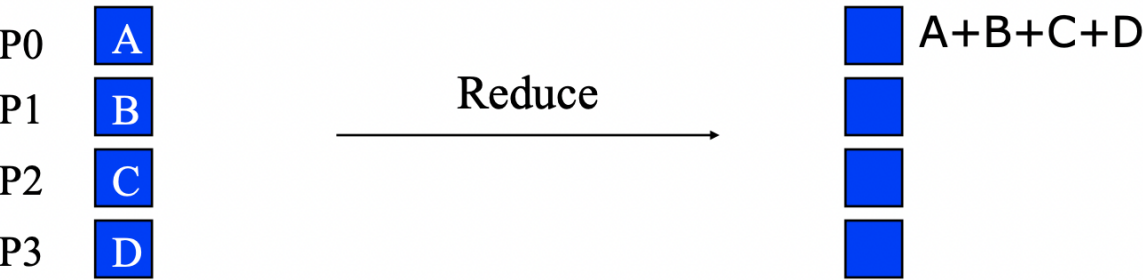
all-gather是多对多通讯， 相当于scatter + broadcast组合：



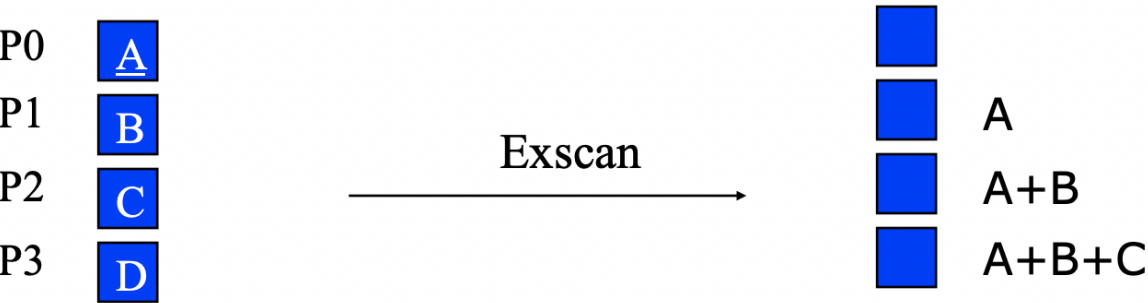
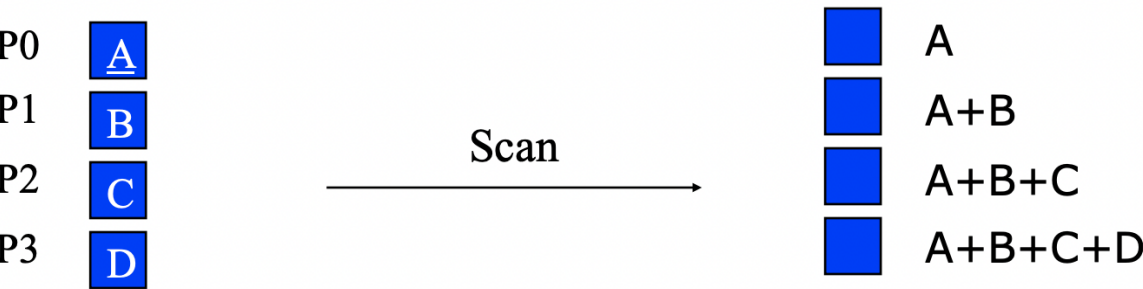
all-to-all是多对多通讯， 相关于gather + broadcast组合：



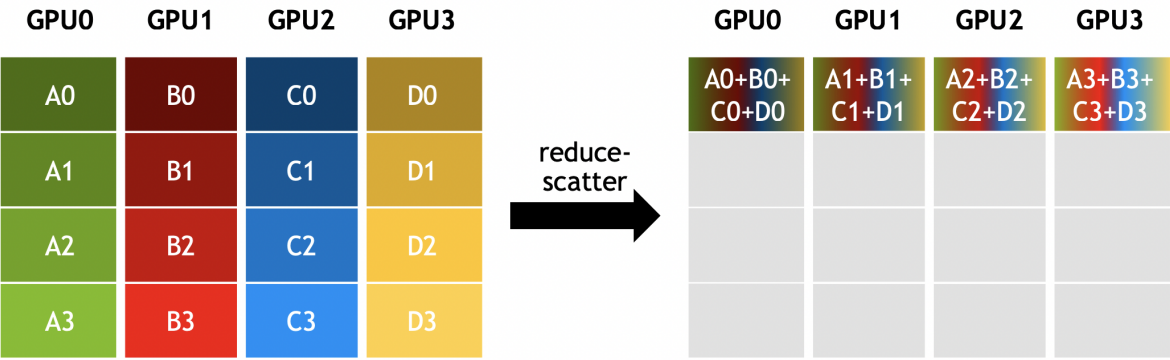
reduce和all-reduce 含义如下，Reduce从多个sender那里接收数据，最终combine到一个节点上，AllReduce从多个sender那里接收数据，最终combine到每一个节点上



scan和ex-scan含义，浏览增加和浏览排除self。



reduce-scatter模式：

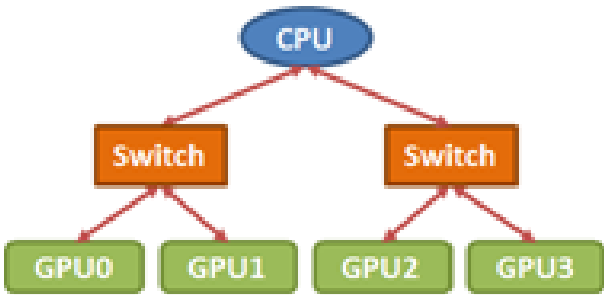


文档参考：

- https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/zh_cn/
- <https://wgropp.cs.illinois.edu/courses/cs598-s15/lectures/lecture29.pdf>
- <https://images.nvidia.com/events/sc15/pdfs/NCCL-Woolley.pdf>

1.2 网络拓扑&基于环的集群（ring-base collectives）

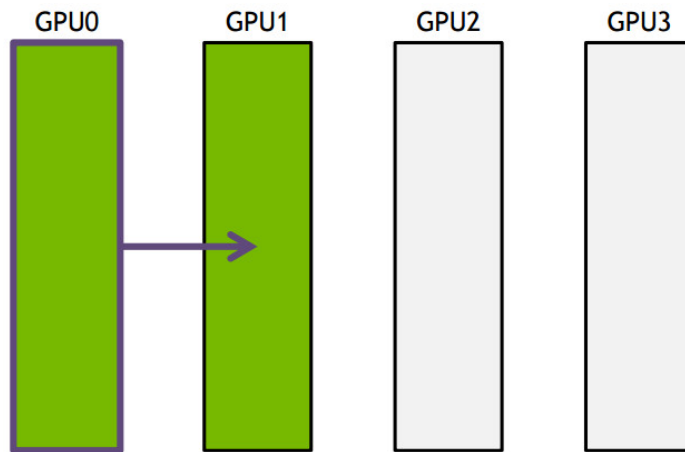
传统集群通讯假设通信节点组成的topology是一颗fat tree，这样通信效率最高。但实际的通信topology可能比较复杂，并不是一个fat tree。因此一般用ring-based Collective communication。单机多卡之间有不同的拓扑结构，影响着数据之间的传输效率。



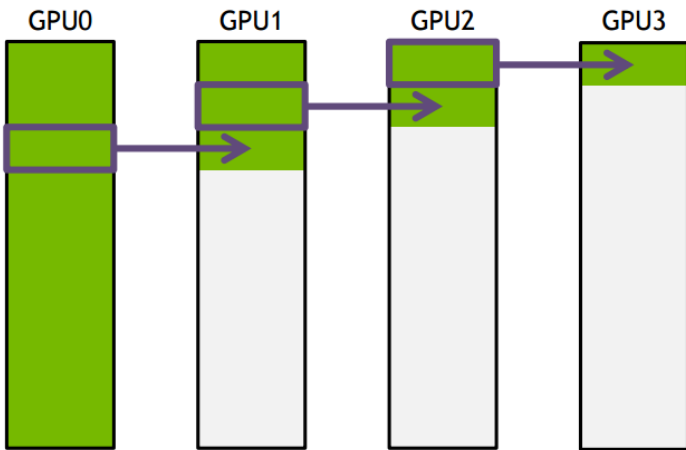
上图展示了一种常见的拓扑结构。以GPU0想向其他卡broadcast数据为例，数据传输先从GPU0 to GPU2， 然后GPU0 to GPU1， GPU2 to GPU3的效率比先GPU0 to GPU1 ， 然后GPU 0 to GPU 2, GPU 1 to GPU 3效率更高， 因为后者的第二部需要竞争CPU。

ring-base collectives将所有的通信节点通过首尾连接形成一个单向环， 数据在环上依次传输。以broadcast为例， 假设有4个GPU， GPU0为sender将信息发送给剩下的GPU， 按照环的方式依次传输：

GPU0 → GPU1 → GPU2 → GPU3



若数据量为N，带宽为B，整个传输时间为 $(K-1) N/B$ 。时间随着节点数线性增长，不是很高效。下面把要传输的数据分成S份，每次只传N/S的数据量，传输过程如下所示：

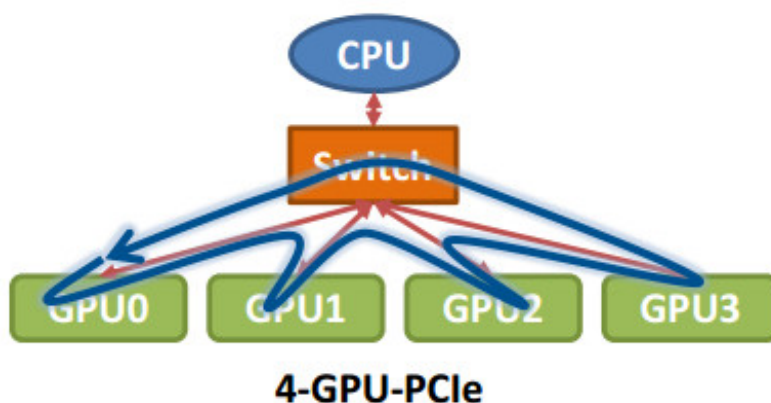


GPU1接收到GPU0的一份数据后，也接着传到环的下个节点，这样以此类推，最后花的时间如下公式，条件是S远大于K，即数据的份数大于节点数，这个很容易满足。所以通信时间不随节点数的增加而增加，只和数据总量以及带宽有关。

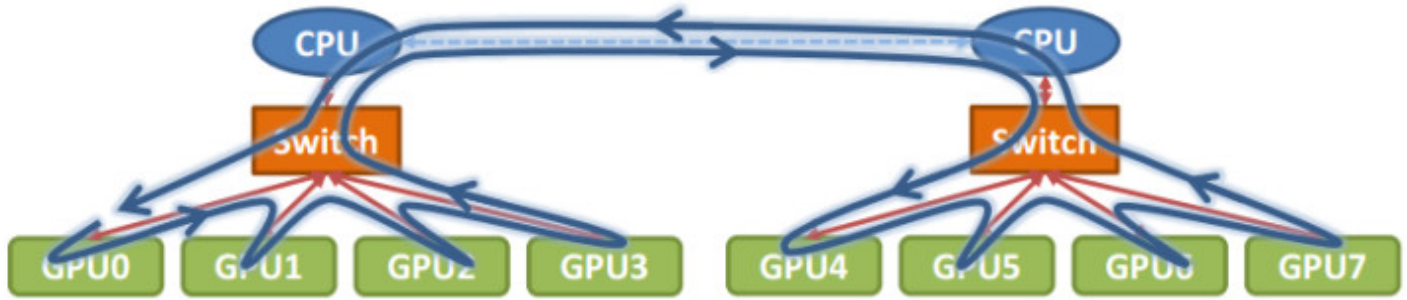
$$S * (N/S/B) + (k-2) * (N/S/B) = N(S+K-2)/(SB) \rightarrow N/B$$

环形是一种很好的方法来优化broadcast，且即使当应用于“树形”PCIe拓扑时，环算法也为几乎所有的标准集合操作提供接近最优的带宽。但请注意，**选择正确的振铃顺序仍然很重要。**

那么在以GPU为通信节点的场景下，怎么构建通信环呢？如下图所示，单机4卡通过同一个PCIe switch挂载在一棵CPU的场景：



单机8卡通过两个CPU下不同的PCIe switch挂载的场景：



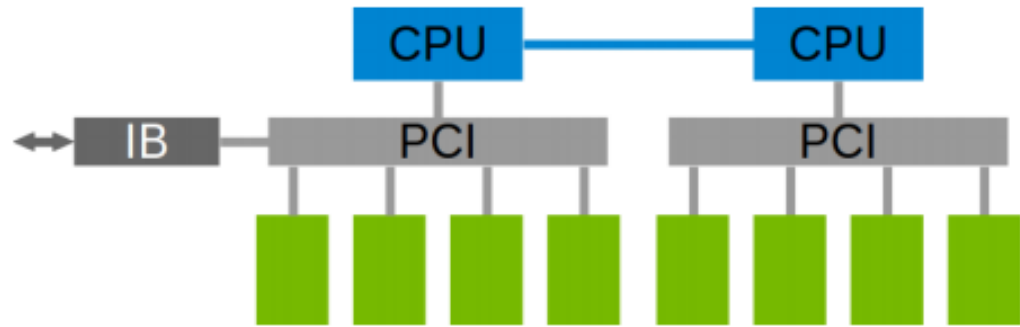
1.3 NCCL实现

NCCL实现成CUDA C++ kernels，包含3种primitive operations：Copy，Reduce，ReduceAndCopy。目前NCCL 1.0版本只支持单机多卡，卡之间通过PCIe、NVlink、GPU Direct P2P来通信。NCCL 2.0会支持多机多卡，多机间通过Sockets (Ethernet)或者InfiniBand with GPU Direct RDMA通信。

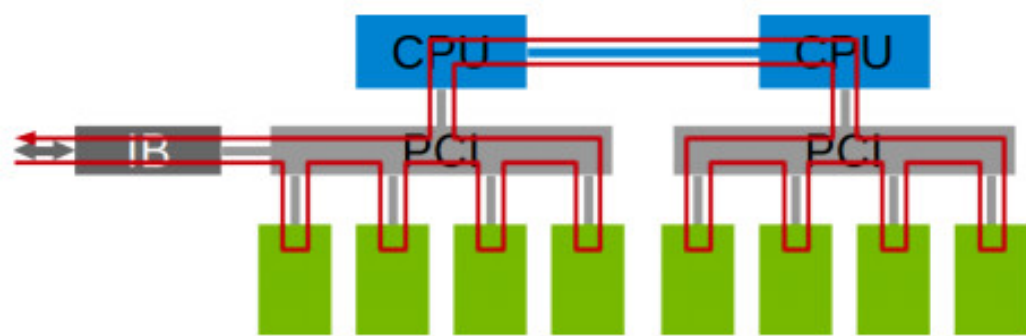
在实现上，NCCL的特性包括：

1. NCCL内collectives的实现方式为ring-style，以提供最大带宽。
2. NCCL能够自动对GPU进行编号并实现最佳ring顺序。
3. 许多collectives实现时需要一个缓冲区来存储中间结果。为了最小化每个GPU上的几MB的内存开销，NCCL将大型集合分成许多小块。对于集合算法的每个步骤和块。
4. 启动单独的内核和cudaMemcpy调用是非常低效的。对于一个collectives算法，将每一个步骤实现为一个kernel，每一个chunk数据都执行cudaMemcpy，这种做法是非常低效的。因此，NCCL内的每一个collective操作都由一个大kernel实现。
5. NCCL广泛使用GPUDirect peer to peer直接访问方式在处理器之间传输数据。

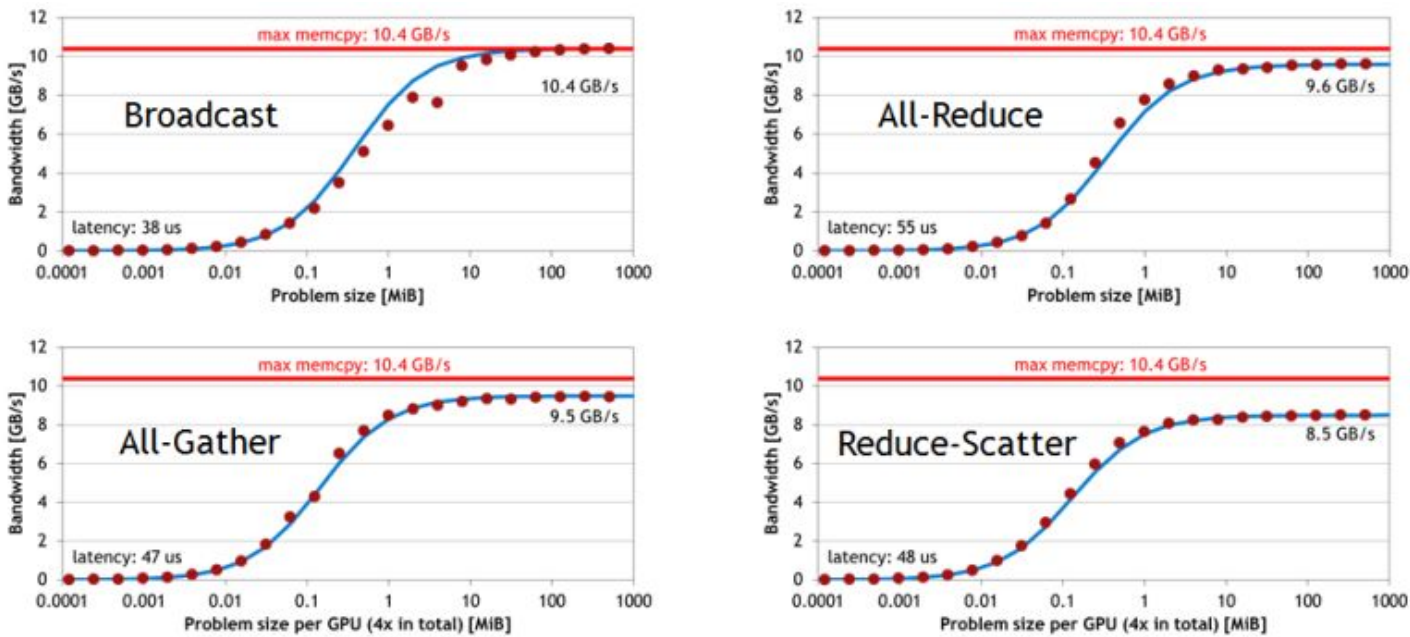
下图所示，单机内多卡通过PCIe以及CPU socket通信，多机通过InfiniBand通信。



多机场景下，也是构成一个环：



下面是单机 4 卡（Maxwel GPU）上各个操作随着通信量增加的带宽速度变化，可以看到带宽上限能达到10GB/s，接近PCIe的带宽。



2.分布式并行训练

2.1 并行训练基础

- **数据并行（DP）**：数据并行是指对训练数据做切分，被多个相同的模型并行处理，一般按照batch 维度切分将数据分配到各个计算单元（worker）中。

- 数据并行训练加速比最高，但要求每个设备上都备份一份模型，显存占用比较高。
- **模型并行 (MP)**：Layer内并行，将模型做切分成子网络，各个不同的子网络并行处理每一条数据。当一个神经元的输入来自另一个训练单元上的神经元的输出时，产生通信开销。
 - 模型并行带来的通信开销和同步消耗超过数据并行，适合在机器内做模型并行。
- **流水线并行 (PP)**：Layer间并行，属于时间上的并行技术，在一个单元计算时，其他计算单元同时计算非依赖任务。
 - 训练设备容易出现空闲状态，加速效率没有DP高；但能减少通信边界支持更多的层数，适合在机器间使用
- **混行并行 (Hybrid Parallel)**：覆盖数据并行、模型并行、流水线并行。

2.2 Paddle分布式训练

Paddle的分布式API大多集中在paddle.fluid下，从Paddle Fluid Release 1.5.1 开始，官方推荐使用fluid下封装的更高级的Fleet API进行分布式训练，Fleet API支持pserver模式和NCCL2模式下的分布式训练。pserver模式即参数服务器模式；NCCL2模式，即集合通信模式（利用NCCL进行通信）；通常，在分布式多GPU的环境下，我们使用NCCL模式的分布式训练。

Paddle分布式API中顶层的库(以集合通信为例)主要包括：paddle.fluid、paddle.fluid.incubate.fleet.collective.fleet等，这些顶层库下面又包含了分布式策略、role_maker、分布式optimizer等相关的接口和方法如：

- 分布式策略相关接口：paddle.fluid.incubate.fleet.collective.DistributedStrategy
- role_maker相关接口：paddle.fluid.incubate.fleet.base.role_maker
- optimizer相关接口：fluid.optimizer
- 分布式optimizer封装接口：fleet.distributed_optimizer

RoleMaker是Paddle分布式中的重要概念，用于描述集群节点的角色信息，目前支持：MPISymetricRoleMaker、PaddleCloudRoleMaker、UserDefinedRoleMaker三种模式。

其中MPISymetricRoleMaker会假设每个节点启动两个进程，1worker+1pserver，这种RoleMaker要求用户的集群上有mpi环境；PaddleCloudRoleMaker是一个高级封装，支持使用paddle.distributed.launch或者paddle.distributed.launch_ps启动脚本；UserDefinedRoleMaker允许用户自定义节点的角色信息，IP和端口信息。

文档参考：<https://www.paddlepaddle.org.cn/tutorials/projectdetail/487871>